10. Protocols and Extensions

Task 1:

Code:

```
protocol Payable {
    var name: String { get }
    func monthlyPay() -> Double
}
extension Payable {
    func payslip() -> String {
        return "\(name)'s Monthly Pay: ₹\(monthlyPay())"
    }
}
struct FullTimeEmployee: Payable {
    let name: String
    let annualSalary: Double

    func monthlyPay() -> Double {
        return annualSalary / 12
    }
}
struct Intern: Payable {
    let name: String
    let monthlyStipend: Double
    func monthlyPay() -> Double {
        return monthlyStipend
```

```
    }
}
struct Contractor: Payable {
    let name: String
    let hourlyRate: Double
    let hours: Double
    func monthlyPay() -> Double {
        return hourlyRate * hours
    }
}
let employee = FullTimeEmployee(
    name: "Asha",
    annualSalary: 600000
)
let intern = Intern(
    name: "Ravi",
    monthlyStipend: 15000
)
let contractor = Contractor(
    name: "Kiran",
    hourlyRate: 500,
    hours: 80
)
let workers: [any Payable] = [
    employee,
```

```
    intern,
    contractor
]
for worker in workers {
    print(worker.payslip())
}
```

Output:

```
Output

Asha's Monthly Pay: ₹50000.0
Ravi's Monthly Pay: ₹15000.0
Kiran's Monthly Pay: ₹40000.0
```

Task 2:

Code:

```
extension Int {
    var isEven: Bool {
        return self % 2 == 0
    }
    func squared() -> Int {
        return self * self
    }
}
print(4.isEven)
print(5.squared())
```

```
extension Array where Element == Int {
    var sum: Int {
        return self.reduce(0, +)
    }
}

let numbers = [10, 20, 30]
print(numbers.sum)
extension String {
    var wordCount: Int {
        return self.split { $0.isWhitespace }.count
    }
}
let sentence = "Swift is easy to learn"
print(sentence.wordCount)
struct Version: Comparable {
    let major: Int
    let minor: Int
    static func < (lhs: Version, rhs: Version) -> Bool {
        if lhs.major != rhs.major {
            return lhs.major < rhs.major
        }
        return lhs.minor < rhs.minor
    }
}
```

```
let v1 = Version(major: 1, minor: 9)
let v2 = Version(major: 2, minor: 0)
print(v1 < v2)
let versions = [
    Version(major: 2, minor: 1),
    Version(major: 1, minor: 9),
    Version(major: 1, minor: 5),
    Version(major: 2, minor: 0)
]
let sortedVersions = versions.sorted {
    $0 < $1
}
for version in sortedVersions {
    print("\(version.major).\(version.minor)")
}
```

Output:

```
Output

true
25
60
5
true
1.5
1.9
2.0
2.1
```

Task 3:

Code:

```
protocol Flyable {
    func fly()
}
protocol Swimmable {
    func swim()
}
struct Duck: Flyable, Swimmable {
    func fly() {
        print("Duck is flying")
    }
    func swim() {
        print("Duck is swimming")
    }
}
struct Eagle: Flyable {
    func fly() {
        print("Eagle is flying")
    }
}
struct Fish: Swimmable {
    func swim() {
        print("Fish is swimming")
    }
```

```
}
func showOff(_ animal: Flyable & Swimmable) {
    animal.fly()
    animal.swim()
}
let duck = Duck()
let eagle = Eagle()
let fish = Fish()
duck.fly()
duck.swim()
eagle.fly()
fish.swim()
showOff(duck)
```

Output:

